

KFOLD: Scalable and Exact Object-Selection Conditions for Kbuild Makefiles

ANONYMOUS AUTHOR(S), Anonymous Institution, USA

Kbuild Makefiles decide which objects enter the build of Linux and other configurable systems. A maintainer who asks why a file was not compiled, or which options would compile it, needs the option condition under which each object is built, but running Make reveals only the objects of one configuration. Computing these conditions statically requires following assignments whose destination depends on an option, overwrites that apply in only some configurations, composite objects, included fragments, directories reached along several paths, and objects that Kbuild builds only because a rule needs them. Executing a Makefile symbolically with one state per branch combination does not scale: on a generated Makefile with 16 independent options, a forking executor tracks more than half a million states. We present KFOLD, which executes Kbuild Makefiles with a Boolean guard on every word of every variable. Appends add guarded words, a conditional overwrite re-guards the words it replaces, and the two arms of a conditional merge into one state at `endif`, so the state stays linear in the number of words. A directory worklist propagates to each Makefile only the part of an incoming guard not already covered, and a closure over rules adds the objects that selected files depend on.

We evaluate KFOLD on Linux v6.6 and four other Kbuild-based systems by evaluating its conditions under concrete configurations and comparing the predictions with the objects of real builds and with GNU Make's own expansion of each Makefile. On four Linux configurations, from `tinyconfig` to `allmodconfig`, KFOLD's predicted objects are exactly the compiled objects: 489 to 25,949 objects, with no false positive and no miss. Per Makefile, its target lists match GNU Make for all 922 to 12,605 active targets. KFOLD analyzes the 2,022 Makefile instances of the Linux x86 tree in 34 s and 162 MB, and its time stays flat on generated Makefiles where the forking executor times out. These results show that per-word guards, together with a faithful model of Kbuild's build semantics, yield object-selection conditions that are both scalable and exact at kernel scale.

CCS Concepts: • **Software and its engineering** → **Software verification**.

Additional Key Words and Phrases: Kbuild, build systems, variability, symbolic execution, object selection

1 Introduction

The build system determines which source files reach the compiler. In the Linux kernel and related projects, Kbuild expresses this choice through Make variables such as `obj-$(CONFIG_USB)` and through conditionally selected subdirectories. A concrete build reveals the objects selected by one `.config`; it does not say which option settings would select a different object. A build condition for every object would let a maintainer explain why a source file was not compiled, compare build rules across releases, or search for a configuration that enables a given target. Analyses of configuration-dependent compilation errors and unmet dependencies already consume such conditions [5, 14].

Recovering these conditions requires more than collecting the `CONFIG_*` names that appear near an object. Consider two consecutive lines of a Kbuild Makefile:

```
obj-$(CONFIG_NET) += net_core.o    obj-$(CONFIG_FAST) := fast_net.o
```

The first line appends `net_core.o` to `obj-y` when `CONFIG_NET=y`. The second computes its destination from `CONFIG_FAST`: when that option is `y`, it *replaces* `obj-y`; otherwise it assigns an unselected variable such as `obj-` and leaves `obj-y` alone. The correct condition for `net_core.o` is therefore `CONFIG_NET ∧ ¬CONFIG_FAST`. Further conditions come from `ifeq` blocks, from composite objects whose members are listed in other variables, from fragments included with `include`, from the

Author's Contact Information: Anonymous Author(s), Anonymous Institution, USA.

2026. ACM XXXX-XXXX/2026/9-ART
<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

chain of parent directories through which Kbuild reaches a Makefile, and from rules: the Linux boot image, real-mode trampoline, and vDSO consist of objects that no target list names and that Kbuild builds only because a rule needs them.

Symbolic execution is a natural way to compute such conditions, because it evaluates Make’s assignments and expansions as written while tracking the configurations under which each result holds. Its classic form keeps one state per path. An earlier symbolic executor for Kbuild Makefiles [13] forks a state at each conditional and merges only identical paths. Kbuild Makefiles, however, consist largely of independent conditional assignments, and each one that changes a value can double the number of distinct states. On a generated Makefile with 16 independent `ifeq` blocks that each append to a helper list, that executor tracks 524,289 states and needs 185 s and 1.7 GB; with 20 blocks it does not finish within five minutes (Section 5.6). The Linux x86 tree has 2,022 Makefile instances with thousands of such assignments.

KFOLD starts from the observation that Kbuild object lists are used as *sets*: an object is either in `obj-y` or it is not. KFOLD therefore attaches a guard to each word rather than keeping one state per configuration. In the example, `obj-y` holds `net_core.o` under `CONFIG_NET` $\wedge$ $\neg$ `CONFIG_FAST` and `fast_net.o` under `CONFIG_FAST`. A conditional overwrite conjoins the negation of its own guard onto the words it may replace. For an `ifeq` block, KFOLD executes each arm on a copy of the state and merges the copies at `endif`, so one state continues after every conditional and k guarded appends yield k guarded words. Across directories, a worklist records for each Makefile the disjunction of guards already propagated to it and processes only the uncovered part of a new guard. On top of this representation, KFOLD models the parts of Kbuild’s build semantics that decide which object files exist: composite members, the rule that built-in objects need a built-in route to their directory, host and user programs, and a closure over rules and sub-makes.

A scalable representation is useful only if its conditions are right, and the strongest evidence for that is agreement with real builds. We therefore evaluate KFOLD’s conditions under concrete configurations against two references: the `.o` files that a build of that configuration produces, and GNU Make’s expansion of the target lists in each traversed Makefile. The subjects are Linux v6.6 under four configurations, BusyBox, coreboot, Barebox, and Das U-Boot. On all four Linux configurations, KFOLD’s predicted set equals the set of compiled objects, from 489 objects for `tinyconfig` to 25,949 for `allmodconfig`. Per Makefile, its target lists agree with GNU Make on every one of 922 to 12,605 active targets. KFOLD analyzes the Linux x86 tree in 34 s and 162 MB. On the same tree, KMAX [3], a static analysis that also carries configuration conditions on values, needs 86 minutes, fails on three directories that exceed a 12 GB memory limit, and misses up to 7,902 compiled objects that it extracts.

This paper makes the following contributions:

- (1) A per-word guarded representation of Make variables, with guarded append, conditional overwrite, and branch merge operations for Kbuild-style assignments with computed names, and an expansion that preserves Make’s semantics for word lists, partially defined variables, substitution references, and deferred appends (Sections 3.3 to 3.5).
- (2) A conditional directory worklist that propagates only uncovered guards, static splicing of included fragments, and a model of Kbuild’s build semantics, including built-in routes and a closure over rules and sub-makes (Sections 3.2, 3.6 and 3.8).
- (3) KFOLD, an implementation that supports the target families, composite objects, and top-level entry points of five Kbuild dialects, and a web interface for querying Linux conditions (Sections 3.7 and 4).

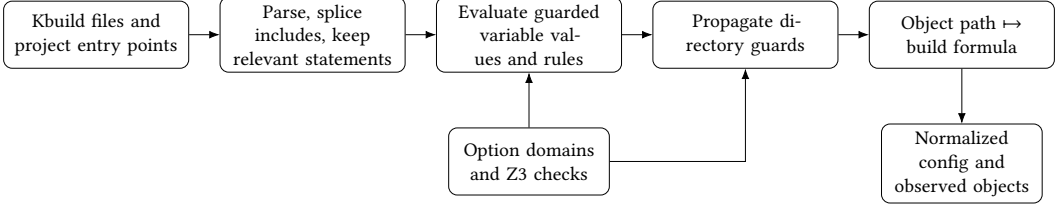


Fig. 1. KFOLD computes one build formula per object path without choosing a configuration. A normalized configuration and the observed object inventory enter only afterward, to evaluate and validate the formulas.

- (4) An evaluation against physical builds and GNU Make on five systems, with exact agreement on four Linux configurations, a scalability comparison with a forking symbolic executor and with KMAX, and an ablation of each design element (Section 5).

2 Overview

Figure 1 shows how KFOLD turns Kbuild files and project entry points into a formula for each object path. KFOLD first parses each Makefile, splices in the fragments it includes, and removes statements that cannot affect target variables or rules. It then executes the remaining statements over guarded variable values, propagates directory guards from parent to child Makefiles, and finally collects guarded object paths: targets, composite members, program objects, and the prerequisites that rules need. A separate evaluator substitutes a concrete configuration into the formulas and compares the predictions with observed objects. The next subsections follow one Makefile through these stages; Section 3 generalizes each operation.

2.1 Kbuild selection in brief

Kbuild uses GNU Make variables to describe objects. A line such as `obj-$(CONFIG_NET) += net.o` computes the destination variable name from an option. If the option expands to `y`, the word is appended to `obj-y` and linked into the kernel; `m` selects `obj-m` for a loadable module; an unset option produces `obj-`, which Kbuild ignores. A word ending in `/` asks Kbuild to descend into that subdirectory’s Makefile. A target such as `driver.o` may be a composite assembled from the objects in `driver-y` or `driver-objs`, and a Makefile may pull further assignments from another file with `include`. The condition for an object therefore combines a local selection condition with a reachability condition inherited from its parent directories.

GNU Make also distinguishes immediate assignment with `:=` from deferred assignment with `=`. The former expands its right-hand side at assignment time; the latter stores an expression that is expanded when the variable is referenced. `+=` follows the variable’s existing flavor, and `?=` assigns only when the variable is undefined. A reference to a variable that is not set in the current configuration expands to the empty string. These rules decide which assignment wins and what a reference produces, and so affect object lists even when a Makefile contains no explicit conditional.

KFOLD takes a source tree, per-project settings that name the entry directories and target-variable families, and a Boolean or tristate option domain. It outputs a set of object paths, each with a formula over those options.

Listing 1 is a simplified Kbuild Makefile that extends the two lines from Section 1. It combines an assignment to a computed variable name, an overwrite, a conditional branch, and a composite target. For this example, assume that each option is either `y` or `unset`. We want a condition for each object, not a list for one chosen configuration.

Listing 1. Simplified Kbuild Makefile. Each object’s final condition also includes the guard under which this directory is entered.

```

1 obj-$(CONFIG_NET) += net_core.o
2 obj-$(CONFIG_FAST) := fast_net.o
3 ifeq ($(CONFIG_WIDE),y)
4     WIDTH := 64
5 else
6     WIDTH := 32
7 endif
8 obj-y += driver_$(WIDTH).o
9 fast_net-y += accelerator.o

```

Table 1. Guarded words after each statement of Listing 1. The last column assumes that the directory is reached under guard D . The map records membership, not word order or multiplicity.

Line	Step	State change	Final contribution
1	Append	<code>net_core.o</code> : N	$D \wedge N \wedge \neg F$
2	Conditional overwrite	<code>fast_net.o</code> : F ; <code>net_core.o</code> : $N \wedge \neg F$	$D \wedge F$
3–7	Conditional	<code>WIDTH</code> : 64 under W , 32 under $\neg W$	—
8	Append	<code>driver_64.o</code> : W	$D \wedge W$
8	Append	<code>driver_32.o</code> : $\neg W$	$D \wedge \neg W$
9	Composite	<code>accelerator.o</code> belongs to <code>fast_net.o</code>	$D \wedge F$

2.2 A trace from assignments to object conditions

Table 1 records the state after each statement of Listing 1. We write N for `CONFIG_NET=y`, F for `CONFIG_FAST=y`, and W for `CONFIG_WIDE=y`. Line 1 adds one word, `net_core.o`, with guard N . Line 2 assigns `obj-y` only when F holds, so `KFOLD` adds `fast_net.o` under F and re-guards the existing word: `net_core.o` survives under $N \wedge \neg F$. Treating the overwrite as unconditional would lose `net_core.o`; ignoring it would keep `net_core.o` under $N \wedge F$, where Make has replaced the value.

Lines 3–7 assign `WIDTH` in both arms of a conditional. `KFOLD` executes the then-arm on one copy of the state under W and the else-arm on another under $\neg W$. At `endif`, it merges the copies: `WIDTH` holds 64 under W and 32 under $\neg W$, and execution continues with one state. Line 8 expands `driver_$(WIDTH).o` into two words with complementary guards. Line 9 defines a composite: its parent `fast_net.o` is selected under F , so its unconditional member `accelerator.o` inherits F . Finally, `KFOLD` conjoins every local condition with the guard D under which the directory is reached, which gives the last column of Table 1.

A forking executor would instead carry one environment for each combination of N , F , and W that produces a different `obj-y`, and every further independent conditional that changes the list can double that number. `KFOLD`’s state after line 9 holds five guarded object words, and a further independent append adds one more.

2.3 The same object through two directories

Directory guards need the same care as variable guards. Suppose a root Makefile selects `a/` under option A and `b/` under option B , and both directories select `shared/`. If `shared/Makefile` selects `x.o`, the condition for `x.o` is $A \vee B$. Marking `shared/` as visited after the A path would lose the B contribution. `KFOLD` instead records, for each Makefile, the disjunction of the guards already propagated to it. When B arrives, it processes only $B \wedge \neg A$; if A already covered B , it does no

new work. For the same reason, a back edge from `shared/` to `a/` stops once it contributes no new configuration.

3 Technique

3.1 Setting and algorithm

Let M be a collection of Kbuild Makefiles and c an assignment to the option symbols they use. For an object path o , write $S_M(o, c)$ when Kbuild builds o under c . KFOLD computes a formula $\Phi_M(o)$ such that evaluating $\Phi_M(o)$ under c predicts $S_M(o, c)$. A variable's state $\sigma[n]$ is a map from words to formulas: an entry $w \mapsto \phi$ says that w belongs to variable n in configurations satisfying ϕ . We write $\sigma[n](w) = \perp$ for an absent word and $\sqcup$ for the pointwise disjunction of two such maps.

Algorithm 1 gives the end-to-end procedure. Its input is the source tree, the entry Makefiles E with their guards, and project settings Γ that name target-variable families and option domains; its output is the map Φ from object paths to formulas. Lines 3–8 take a Makefile m with incoming guard γ and keep only the part δ not already covered by $R[m]$. Lines 9–11 parse m , splice its includes, reduce it, and execute it once with EXEC (Algorithm 2); line 12 restricts the cached state to δ , and line 13 records the restricted instance. Lines 14–16 queue the child directories selected in that instance. After the queue empties, lines 18–23 collect targets, composite members, and program objects from all instances and disjoin repeated contributions, and line 24 adds the objects that rules need (Section 3.8). A missing entry file contributes no edge, and a file that fails to parse contributes an empty state and is reported.

Algorithm 2 executes one statement s on state σ under ambient guard g ; a statement list is executed in order. Lines 1–10 handle an assignment: line 2 expands the right-hand side into guarded words, line 3 expands the left-hand side into guarded variable names, and lines 5–9 either overwrite or append. Lines 11–16 handle a conditional: line 12 evaluates the test and copies the state, line 13 executes both arms on the copies, and lines 14–16 merge the copies back into σ . Rules and sub-make recipes are recorded as guarded lists as well (Section 3.8); include directives have been replaced by the statements they name.

The two algorithms connect through the overview example. For Listing 1, $C[m]$ holds the two guarded WIDTH words; restricting that state by the parent guard D at line 12 of Algorithm 1 produces $D \wedge W$ and $D \wedge \neg W$ for the two driver objects. In the shared-directory example, `shared/Makefile` is processed first under A and then under $B \wedge \neg A$, and line 21 yields $A \vee B$ for `x.o`.

The observation unit is an object path, not a source file. A path can denote a compiler output, a module's composite object, a member of a composite, an object of a host program, or a link artifact. Section 5 maps a path to a physical object when the normalized spellings match.

3.2 Parsing, include splicing, and dependence reduction (Algorithm 1, line 10)

KFOLD reads a Makefile, normalizes line endings, and parses it with `pymake3`. Kbuild evaluates an included fragment in the context of the including Makefile, with the source tree as working directory. KFOLD mirrors this by *splicing*: it replaces each include whose path it can resolve with the statements of the named file, recursively and without cycles, before any further processing. Paths are resolved statically from `src`, `obj`, `src tree`, the configured fixed definitions, earlier assignments in the same file whose values are constant, and `addprefix` and `addsufffix` calls over resolved arguments. The Linux AMD GPU driver, for example, defines `FULL_AMD_PATH := $(src tree)/$(src)/..`, includes `$(FULL_AMD_PATH)/amdkfd/Makefile`, and builds the list of display-library Makefiles to include with `addprefix` and `addsufffix`; after splicing these 51 files, the composite `amdgpu.o` has 786 members. An include whose path remains unresolved stays in place and contributes nothing, and KFOLD counts it.

Algorithm 1 Extract guarded Kbuild object conditions

Require: source tree M , guarded entry files E , project settings Γ

Ensure: object-path map Φ

```
1:  $R[m] \leftarrow \perp$  for each file  $m$ ;  $Q \leftarrow E$ 
2:  $C \leftarrow$  empty map;  $I \leftarrow \emptyset$ 
3: while  $Q$  is nonempty do
4:   remove  $(m, \gamma)$  from  $Q$ 
5:    $\delta \leftarrow \gamma \wedge \neg R[m]$ 
6:   if  $\delta$  is unsatisfiable then
7:     continue
8:   end if
9:   if  $C[m]$  is undefined then
10:     $C[m] \leftarrow \text{EXEC}(\text{reduce}(\text{splice}(\text{parse}(m))), \sigma_\Gamma, \Gamma)$ 
11:   end if
12:    $S \leftarrow \text{restrict}(C[m], \delta)$ 
13:    $R[m] \leftarrow R[m] \vee \gamma$ ; add  $S$  to  $I$ 
14:   for each selected directory  $(d, \eta)$  in  $S$  do
15:     add  $(\text{entry}(d), \eta)$  to  $Q$  if its file exists
16:   end for
17: end while
18:  $\Phi \leftarrow \emptyset$ 
19: for each restricted instance  $S \in I$  do
20:   for each target, member, or program object  $(o, \phi) \in \text{objects}(S, \Gamma)$  do
21:      $\Phi[o] \leftarrow \Phi[o] \vee \phi$  (initially  $\perp$ )
22:   end for
23: end for
24:  $\Phi \leftarrow \text{close}(\Phi, I, \Gamma)$  ▷ rule prerequisites and sub-makes
25: return  $\Phi$ 
```

A preliminary dependency pass then identifies the variables that can influence target families, composites, programs, and rules, and removes the statements that cannot. EXEC runs on the reduced list against a fresh state σ_Γ that holds `src`, `obj`, and the configured definitions. The reduction keeps production Makefiles tractable, because most of their statements set compiler flags that do not affect which objects exist. A file that cannot be parsed contributes an empty state and is reported; in the evaluation this occurs for one coreboot file.

3.3 Guarded assignment (Algorithm 2, lines 1–10)

Consider an assignment whose left-hand side expands to name n under guard ψ (lines 3–4). An append adds each expanded right-hand word under the conjunction of ψ and the word’s own expansion guard (line 8). An overwrite replaces n where ψ holds: new words receive $\psi \wedge \phi_w$, and an old word guarded by ϕ survives under $\phi \wedge \neg\psi$ (line 6). If an old and a new contribution share a word, $\sqcup$ disjoins their guards. Line 2 of Listing 1 exercises this case: $\psi = F$, so `net_core.o` goes from N to $N \wedge \neg F$. The same operation handles a guard that comes from the computed name rather than from an enclosing `ifeq`.

KFOLD follows GNU Make’s flavors. For `:=` it expands the right-hand side at assignment time. For `=` and `?=`, and for `+=` on a variable that is undefined or recursive, it stores text that contains a reference for later expansion and splits text without references into words directly (line 2);

Algorithm 2 EXEC: execute one statement over guarded words

Require: statement s , state σ , ambient guard g

Ensure: updated state σ

```
1: if  $s$  is an assignment  $x \text{ op } e$  then
2:    $V \leftarrow \{w \mapsto \phi_w\}$  from  $\text{expand}(e, \sigma)$ , or  $\{e \mapsto \top\}$  if  $\text{op}$  defers  $e$  and  $e$  has a reference
3:   for each guarded name  $(n, \psi_n) \in \text{expand}(x, \sigma)$  do
4:      $\psi \leftarrow g \wedge \psi_n$ 
5:     if  $\text{op} \in \{=, :=\}$  or  $\sigma[n]$  is undefined then
6:        $\sigma[n] \leftarrow \{w \mapsto \psi \wedge \phi_w\} \sqcup \{w \mapsto \phi \wedge \neg\psi : (w \mapsto \phi) \in \sigma[n]\}$ 
7:     else
8:        $\sigma[n] \leftarrow \sigma[n] \sqcup \{w \mapsto \psi \wedge \phi_w\}$ 
9:     end if
10:  end for
11: else if  $s$  is a conditional with test  $t$ , then-arm  $T$ , else-arm  $F$  then
12:    $\kappa \leftarrow \text{cond}(t, \sigma)$ ;  $\sigma_t \leftarrow \text{copy of } \sigma$ ;  $\sigma_e \leftarrow \text{copy of } \sigma$ 
13:   EXEC( $T, \sigma_t, g \wedge \kappa$ ); EXEC( $F, \sigma_e, g \wedge \neg\kappa$ )
14:   for each name  $n$  assigned in  $T$  or  $F$  do
15:      $\sigma[n](w) \leftarrow g \wedge \phi$  if  $\sigma_t[n](w) = \sigma_e[n](w) = \phi$ , else  $(g \wedge \kappa \wedge \sigma_t[n](w)) \vee (g \wedge \neg\kappa \wedge \sigma_e[n](w))$ 
16:   end for
17: end if
18: return  $\sigma$ 
```

Linux's kernel/Makefile, for instance, starts `obj-y = fork.o exec_domain.o . . .`. When a Makefile has been executed, `KFOLD` expands the deferred words of its target, composite, and program lists, because `Kbuild` reads those lists only after the whole file. The difference is visible in `drivers/xen/Makefile`, which appends `$(xen-pad-y)` to `dom0-y` one line *before* it defines `xen-pad-y`; only the deferred reading finds `xen-acpi-pad.o`. Line 5 treats only `=` and `:=` as overwrites, so `?=` and `::=` on an already defined variable take the append path; GNU Make would instead leave the value unchanged for `?=` and replace it for `::=`. Table 2 summarizes these boundaries.

3.4 Computed names and expansion (Algorithm 2, lines 2–3)

In `obj-$(CONFIG_X) += foo.o`, the left-hand side expands to `obj-y`, `obj-m`, or an unselected name under distinct guards. EXEC expands the name before updating the state (line 3) and combines the name guard, ambient guard, and value guard (lines 4, 6, and 8). A single assignment can therefore contribute to several target families. The same expansion produces `driver_64.o` and `driver_32.o` from `driver_$(WIDTH).o` in Listing 1.

An assignment's right-hand side is a *word list*: parts separated by whitespace are independent elements, and only parts written without whitespace between them, such as `driver_$(WIDTH).o`, are concatenated by taking the product of their alternatives. Linux's `arm_scmi` Makefile, for example, defines `scmi-module-objs := $(scmi-driver-y) $(scmi-protocols-y) $(scmi-transport-y)`; treating the value as one concatenation would multiply the three lists' alternatives, while the word-list reading keeps each object's own guard. Names, `ifeq` operands, and function arguments are expanded as strings.

A reference to a variable expands to its guarded words and, where none of them is present, to the empty string. The empty alternative matters inside a longer value. The Makefile of Linux's `dvb-core` driver defines `dvb-net-$(CONFIG_DVB_NET) := dvb_net.o` and then `dvb-core-objs`

Table 2. Make constructs and `kFOLD`’s interpretation.

Construct	Implemented behavior	Boundary
<code>+=, =, :=</code>	Guarded words; GNU Make flavors, deferred lists expanded after the file	Order and duplicates are lost.
<code>?=, ::=</code>	Assign if undefined; otherwise append	Differs from GNU Make when the variable is defined.
<code>ifeq, ifneq, ifdef, ifndef</code>	Copy arms, execute, re-guard, merge	—
References, <code>\$(V:a=b)</code>	Guarded words plus the empty alternative; substitution per word	—
Text functions and <code>call</code>	Guarded alternatives	Concatenations above 64 alternatives are collapsed.
<code>include</code>	Spliced when the path resolves statically	Unresolved paths contribute nothing.
Rules and sub-makes	Prerequisites followed from selected files	Recipes other than sub-makes are not executed.
<code>\$(shell ...)</code>	Concrete subprocess (none in sandbox mode)	Host environment may affect results.

`:= dvbdev.o . . . $(dvb-net-y) . . .`, and without it every member of the composite would depend on that option. Substitution references such as `$(IPA_VERSIONS:%=data/ipa_data-v%.o)` and `ixgbe-$(CONFIG_FCOE:m=y)` apply GNU Make’s pattern substitution to each guarded word. Concatenation drops combinations whose guard simplifies to false, and when more than 64 alternatives remain, it collapses them into one string under an unconditional guard. The expander also handles substitution, filtering, prefix and suffix addition, string tests, positional words, `foreach`, and `call`. A `$(shell ...)` expression runs as a concrete subprocess with a five-second timeout, so the analysis depends on the host shell and tools; the evaluated Linux tree contains 15 such calls.

3.5 Conditional execution and branch merge (Algorithm 2, lines 11–16)

For `ifeq` and `ifneq`, the condition evaluator expands both sides into guarded alternatives; equality holds under the disjunction of the guards of equal pairs (line 12). For `ifdef` and `ifndef`, it checks under which guards the variable’s expansion is undefined. A chain of `else ifeq` arms is a nested conditional in the `else`-arm, so only the first matching arm contributes.

`EXEC` copies the entering state for each arm and executes the copies under $g \wedge \kappa$ and $g \wedge \neg \kappa$ (line 13). The merge at lines 14–16 rebuilds only the variables assigned in either arm. For such a variable, a word with guard ϕ_t in the then-copy and ϕ_e in the else-copy receives $(g \wedge \kappa \wedge \phi_t) \vee (g \wedge \neg \kappa \wedge \phi_e)$. Re-guarding matters for a word that an arm did not touch: suppose `WIDTH` had the value 16 before line 3 of Listing 1. Each copy still carries 16 after its own overwrite is applied only under that arm’s guard; conjoining the then-copy with κ and the else-copy with $\neg \kappa$ removes 16, as it should.

When both copies carry the *same* guard ϕ for a word, the disjunction equals $g \wedge \phi$, and `kFOLD` uses that form directly. The difference is decisive for scale. In a Makefile with k independent `ifeq` blocks that each append one object to `obj-y`, every block assigns `obj-y` in one arm; building the disjunction for the untouched words would nest each earlier word’s guard once per later block, so that after 16 blocks the guard of the first object would be a 16-level nested formula rather than `CONFIG_B0=y`. We checked with Z3 that the two forms give equivalent formulas for all 14,713 Linux targets of an earlier version of the analysis; they differ syntactically for 2,857 of them. Variables assigned in neither arm keep their entering representation.

3.6 Conditional directory reachability (Algorithm 1, lines 3–16)

The worklist starts from the configured entry directories. For a Makefile reached under ρ , KFOLD restricts its local state by ρ ; directory words in target variables, and the words of `subdir-y` and `subdir-m`, produce child paths with guards (line 14). Paths are resolved to canonical filesystem paths, so a syntactic `../` back edge refers to the same Makefile.

A Boolean visited set would lose conditions at a shared child: the first visit may cover A while a later path adds B . KFOLD instead maintains $R[m]$, the disjunction of guards already propagated to m . For an incoming guard γ it computes $\delta = \gamma \wedge \neg R[m]$ (line 5). If δ is unsatisfiable, the visit is skipped (lines 6–8). Otherwise the cached state of m is restricted by δ (line 12), its targets and children receive that condition, and $R[m]$ becomes $R[m] \vee \gamma$ (line 13). Because the source files and option domains are finite, $R[m]$ can grow only finitely often, so the worklist terminates.

Entry points. Kbuild enters its first directories from top-level Makefiles that KFOLD does not execute, because they consist mostly of rules, recipes, and host-tool invocations. In Linux, the root Kbuild selects `arch/$(SRCARCH)/`, the top-level Makefile adds `lib/` through its `libs-y` list, and `arch/x86/Makefile` adds `arch/x86/pci/` through `drivers-$(CONFIG_PCI)` and builds the boot image with $\$(MAKE) \$(build)=arch/x86/boot \text{bzImage}$. The project settings of KFOLD state these entries: directories, each optionally guarded by an option that must be set, with shell globs for $\$(wildcard \dots)$ terms; goals such as `arch/x86/boot/bzImage`; fixed definitions such as `SRCARCH=x86` or the guarded `BITS=64?CONFIG_X86_64`; and the objects of the final link. For each subject we copied these entries from the top-level Makefiles; Section 5.1 gives the counts.

3.7 Target families, composites, and aggregation (Algorithm 1, lines 18–23)

Project settings specify target-variable families. Conventional Kbuild uses `obj-` and `lib-`, and Linux also builds the objects of `extra-y` and `always-y`; Barebox adds `pbl-` and `obj-pbl-` for its pre-bootloader, and coreboot uses one family per firmware stage, such as `bootblock-` and `ramstage-`. A target list is a family prefix followed by `y`, `m`, or nothing. Helper variables that merely share the prefix are not targets; Linux’s `obj-pvrusb2-dvb-y`, for example, is a list that the `pvrusb2` Makefile splices into a composite. coreboot’s special families `all-` and `all_x86-` add their objects to several stages; the settings list these aliases, following the handlers in coreboot’s top-level Makefile. `inc`. The extractor records target words ending in `.o` or `.a`, maps source suffixes to object paths for dialects that list sources, and normalizes `..` segments.

Composite resolution follows the members of each selected target t in $t-y$, $t-objs$, and $t-lib$, recursively, and in $t-m$ when t is a module. A member’s condition is the conjunction of its own condition and its container’s condition, as for `accelerator.o` in Table 1. A composite may list an object of its own name, as in `dim-y := dim.o net_dim.o` of `lib/dim`. Members matter at scale: 14,351 of the 29,245 Linux object paths are composite members, such as the fifteen members of `fs/ext2/ext2.o`. The objects of a multi-object host or user program p listed in `hostprogs` or `userprogs` are the words of $p-objs$. When several instances contribute the same path, line 21 disjoins their conditions.

3.8 Kbuild build semantics (Algorithm 1, lines 18–24)

The lists say which objects are *selected*; which object files Kbuild writes depends on further rules of `scripts/Makefile.build`. KFOLD models four of them, each enabled for Linux by a setting.

Built-in composites. When a composite is built into the kernel, Kbuild links its members into the directory’s archive and writes no object for the container. Only a module’s composite, such as `fs/ext2/ext2.o` under `CONFIG_EXT2_FS=m`, has its own object. KFOLD therefore reports a container only through `obj-m`.

Built-in routes. Kbuild builds a directory’s built-in.a, and with it the directory’s obj-y objects, only if the parent reached the directory through obj-y (the need-builtin flag). A directory reached only through obj-m or subdir-y builds its modules, lib-y, and extra-y objects but not its obj-y objects. Under allmodconfig, for example, drivers/video/fbdev/mmp/ is entered through obj-\$(CONFIG_MMP_DISP) with CONFIG_MMP_DISP=m, so its built-in mmp_ctrl.o is not built even though CONFIG_MMP_DISP_CONTROLLER=y. KFOLD computes, for every Makefile, the guard under which it is reached through obj-y entries only, as a fixpoint over the analyzed instances starting from the entry directories, and conjoins it with the conditions of obj-y objects.

Rule prerequisites. Some objects are in no list and are built only because a rule needs them. In arch/x86/entry/vdso/Makefile, vdso-image-64.o is in obj-y; Kbuild compiles it from the generated vdso-image-64.c, a pattern rule \$(obj)/vdso-image-%.c: \$(obj)/vdso%.so.dbg generates that file, and an explicit rule links vdso64.so.dbg from \$(vobjs), which holds vdso-note.o and the other vDSO objects. KFOLD records each rule’s targets and prerequisites, expanded under their guards, as guarded lists in the state, together with sub-make recipes of the form \$(MAKE) \$(build)=dir goal. The closure at line 24 then starts from every object in Φ and every configured goal and follows explicit rules, pattern rules, the implicit steps from %.o to %.c and %.S, and sub-makes, which it analyzes on demand; every object reached is built under the conjunction of the guards along the way. As in GNU Make, a pattern rule applies only if each prerequisite exists or can be made, so \$(obj)/%-x32.o: \$(obj)/%.o does not demand a nonexistent vdso-image.o. The boot image, the compressed kernel, the real-mode trampoline, and purgatory are reached this way: the first through the goal arch/x86/boot/bzImage, the others through rules and sub-makes in the Makefiles themselves.

Link artifacts. The final link of the kernel writes vmlinux.o, init/version-timestamp.o, .vmlinux.export.o under CONFIG_MODULES, and two .tmp_vmlinux.kallsyms objects under CONFIG_KALLSYMS; the settings list them, following scripts/link-vmlinux.sh.

3.9 Meaning of a formula

A formula ranges over the option values of the run’s domain. The Boolean domain contains y and unset; the tristate domain adds m and conjoins CONFIG_MODULES=y with every m alternative; project settings can add finite domains for other names. Z3 checks guard satisfiability within that domain. Φ_M approximates S_M because the supported language is narrower than GNU Make (Table 2) and because the entry settings stand in for top-level orchestration. Φ_M also omits Kconfig’s depends on, select, choice, and normalization rules. A satisfying assignment of $\Phi_M(o)$ is thus a candidate input for the configurator, not a configuration that Kconfig is known to accept.

4 Implementation

KFOLD is written in Python and uses pymake3 for parsing and Z3 for guards. Option values are finite Z3 enumeration sorts: a two-value sort or, when a project opts in, a three-value sort for y, m, and unset. Guards are built from equalities on these values with conjunction, disjunction, and negation, simplified after each operation; satisfiability queries drive state restriction and worklist propagation. KFOLD serializes each Makefile’s executed state, including its recorded rules, into a per-run cache keyed by canonical path, so a later directory route can restrict the same state under a new guard without parsing the file again (Algorithm 1, line 12). The number of analyzed instances can therefore exceed the number of unique Makefiles. The runner also records the includes spliced or left unresolved and the files that failed to parse, and for each object path the Makefiles and variables or rules that contribute it.

The project settings file lists the entry directories and goals, target families, family aliases, fixed definitions, link artifacts, and option domain, and switches the built-in semantics of Section 3.8.

For Linux it has eight entry directories (the root, lib/, and six arch/x86 directories, four of them guarded), one goal, and the definitions of SRCARCH and BITS. A sandbox mode, used by the web interface, replaces `$(shell ...)` by the empty string and keeps include splicing inside the analyzed tree.

The web interface precomputes the Linux conditions into a table of 74,205 shared formula nodes. A developer can search any of the 29,245 object paths, see its condition and the Makefiles and variables that select it, check it against a pasted .config, list the objects a configuration builds, or analyze a Makefile of their own. On the four evaluated profiles its predictions equal KFOLD’s own.

5 Evaluation

We evaluate whether KFOLD’s conditions predict the objects that concrete builds produce, whether its per-file semantics match GNU Make, how it scales, how it compares with KMAX, and which of its design elements matter. We ask five research questions:

- RQ1 (build agreement)** When the conditions are evaluated under a normalized configuration, how many predicted objects are compiled, and how many compiled objects are predicted?
- RQ2 (Make agreement)** Do KFOLD’s per-file target lists match GNU Make’s expansion of the same Makefiles under the same configuration?
- RQ3 (scalability)** How do KFOLD’s time and memory grow with the number of conditional assignments, compared with a forking symbolic executor and with KMAX, and what does a whole tree cost?
- RQ4 (comparison)** How do KMAX’s conditions compare with KFOLD’s against the same builds?
- RQ5 (ablation)** How much does each design element contribute to build agreement?

5.1 Subjects, configurations, and builds

The subjects are Linux v6.6 at commit `ffc253263a1375a65fa6c9f62a893e9767fbebfa`, BusyBox 1.36.1, coreboot 4.22.01, Barebox 2024.01.0, and Das U-Boot 2024.01. Linux and BusyBox are the two systems in KMAX’s evaluation [3]. Coreboot, Barebox, and U-Boot are firmware and bootloader projects whose Kbuild dialects add families for stages, pre-bootloader code, or special classes. The Linux source has two local edits that add `-std=gnu11` to Makefile flags; their diff hash is recorded in the artifact manifest. Each subject’s entry settings copy the directory lists of its top-level Makefiles: 8 directories and one goal for Linux, 37 directories for U-Boot (the main-build `libs-y` list and `arch/sandbox/Makefile`), 15 for Barebox (`common-y`), 28 for BusyBox (`core-y` and `libs-y`), and 40 patterns for coreboot (`subdirs-y`, with globs for its `$(wildcard ...)` terms).

Table 3 lists each configuration and the status of its physical object inventory. Linux `tinyconfig` is an i386 build; the other Linux profiles are x86-64. Clean GCC 12 rebuilds of the normalized i386 `tinyconfig` and x86-64 `defconfig` reproduced their archived object-path sets exactly, with 489 and 2,874 paths. A fresh Barebox sandbox build likewise reproduced all 406 archived paths. A fresh U-Boot build compiled all 1,123 archived objects and 186 more, but final packaging failed because the Python `libfdt` module was unavailable; we report U-Boot against both inventories. The Debian and `allmodconfig` Linux profiles, BusyBox, and coreboot use archived inventories. The coreboot QEMU i440fx archive comes from a ROM build that did not complete.

5.2 Metrics and reference sets

KFOLD emits an extracted object-path set U and a condition for each path. For a normalized configuration c , evaluating the conditions yields a predicted set $P_c \subseteq U$. The physical inventory B_c

Table 3. Configurations and physical inventories. “Reproduced” means that a fresh build produced the same set of object paths as the archive, not byte-identical objects.

Subject	Configuration	Fresh build status	Inventory
Linux	i386 tinyconfig	Completed with GCC 12	489 paths reproduced
Linux	x86-64 defconfig	Completed with GCC 12	2,874 paths reproduced
Linux	Debian x86-64	Not rebuilt	15,419 archived paths
Linux	x86-64 allmodconfig	Not rebuilt	25,949 archived paths
Barebox	sandbox_defconfig	Completed	406 paths reproduced
U-Boot	sandbox_defconfig	Compilation completed; packaging failed	1,123 archived, 1,309 fresh
BusyBox	defconfig	Not rebuilt	191 archived paths
coreboot	QEMU i440fx	Not rebuilt	674 paths from partial ROM build

is the set of .o paths in the build tree. We define $TP = |P_c \cap B_c|$, $FP = |P_c \setminus B_c|$, $FN_U = |(B_c \cap U) \setminus P_c|$, and $O = |B_c \setminus U|$. We report precision $TP/(TP+FP)$ and overlap $TP/|B_c|$, the fraction of all compiled objects that are predicted. Overlap charges `KFOLD` for every compiled object it does not extract, so it is the strict measure of completeness; FN_U separately counts extracted objects whose condition evaluates to false although the object was compiled. The Linux archives hold only the kernel directories, not tools/ or scripts/; a prediction under a top-level directory absent from an archive is reported separately rather than counted as a false positive.

The second reference invokes GNU Make on each traversed Makefile from the source root, with `src` and `obj` naming its directory, the same fixed definitions, and the options of the same normalized configuration, and reads the expanded target lists. It compares these lists with the lists that `KFOLD` computes for that file alone, before directory guards are applied. This reference checks `KFOLD`’s per-file Make semantics independently of directory reachability and build semantics. To evaluate a formula, the evaluator substitutes the configuration value of every option variable that appears in it, treating an option that is unset or “is not set” as `undef`; GNU Make likewise receives only the options that are set.

5.3 Setup

Experiments ran on an Intel Core i7-7700 with eight logical CPUs and 31 GiB of RAM under Debian Linux, with Python 3.14.7, Z3 4.13.3, and GNU Make 4.4.1; clean Linux builds used GCC 12.5.0. Source revisions, archive hashes where available, configuration hashes, and build statuses are recorded in `evidence/manifest.json`. Each reported time is a single run, so small differences are descriptive only. The Linux analysis uses the tristate domain; the Debian profile, for example, assigns `m` to 3,934 options. `KMAX v4.10` runs under `python3 -O`, with its debug mode also disabled in the processes it spawns; Section 5.7 describes its setup.

5.4 RQ1: Build agreement

On all four Linux profiles, `KFOLD`’s predicted set is exactly the set of compiled objects (Table 4): every predicted object is compiled, and every compiled object is predicted. The predicted set grows from 489 paths for `tinyconfig` to 25,949 for `allmodconfig`; it includes the boot image, real-mode, `vDSO`, `purgatory`, and `EFI-stub` objects, composite members of modules, and the link artifacts. One further prediction lies outside the archives’ scope: under `allmodconfig`, `drivers/nvdimmm/Makefile` builds `$(TEST_SRC)/iomap.o` with `TEST_SRC := ../../tools/testing/nvdimmm/test`, whose object lands in `tools/`, which the archive omits. A fresh build of `drivers/nvdimmm/` under the

Table 4. Predicted paths versus compiled Linux .o paths. FN_U counts compiled extracted paths that are not predicted; O counts compiled paths outside U .

Configuration	Pred.	TP	FP	FN_U	O	Prec.	Overlap
tinyconfig (i386)	489	489	0	0	0	100.0%	100.0%
defconfig (x86-64)	2,874	2,874	0	0	0	100.0%	100.0%
Debian (x86-64)	15,419	15,419	0	0	0	100.0%	100.0%
allmodconfig (x86-64)	25,949	25,949	0	0	0	100.0%	100.0%

Table 5. Physical-path comparison for one configuration per non-Linux subject. U-Boot is shown against the archived and the fresh inventory; coreboot’s archive is from a partial ROM build.

Subject	$ U $	Pred.	TP	FP	FN_U	O	Prec.	Overlap
BusyBox	609	162	162	0	0	29	100.0%	84.8%
coreboot	4,581	540	437	103	0	237	80.9%	64.8%
Barebox	1,384	406	402	4	0	4	99.0%	99.0%
Das U-Boot (archived)	3,148	1,029	952	77	0	171	92.5%	84.8%
Das U-Boot (fresh)	3,150	1,029	1,029	0	0	280	100.0%	78.6%

same configuration produced tools/testing/nvdimmm/test/iomap.o, so this prediction is also correct.

The other dialects show high agreement with fewer build rules modeled (Table 5); their settings do not enable the Linux semantics of Section 3.8. No subject has an extracted object that is compiled but not predicted. All 162 BusyBox predictions are compiled, and its 29 objects outside U are built-in.o intermediates. Barebox reaches 99.0% precision and overlap; its four outside objects are generated default-environment blobs and an image-metadata object. For U-Boot, all 77 predictions missing from the archived inventory appear in the fresh build, so against that build precision is 100%; its outside objects there are 167 built-in.o intermediates, 96 host tools under tools/ and scripts/, and 17 others. Coreboot is the hardest subject. Of its 237 outside objects, 115 are host utilities under build/util/ and 104 belong to the SeaBIOS payload, an external project, so KFOLD predicts 437 of the 455 firmware objects. Its 103 false positives lie in the verstage, smm, and decompressor stages, which the top-level Makefile builds only for boards that need them and which this incomplete ROM build never produced.

5.5 RQ2: Agreement with GNU Make

Per Makefile, KFOLD’s target lists agree exactly with GNU Make (Table 6). Across the traversed Linux Makefiles, GNU Make reports 922, 1,837, 6,201, and 12,605 active targets for the four profiles, and KFOLD predicts exactly these. The agreement covers overwrites with computed names, nested conditionals, \$(BITS)-dependent names, word lists built from partially defined variables, substitution references, deferred appends, and the 118 spliced includes. It does not cover directory reachability or build semantics, which RQ1 tests.

5.6 RQ3: Scalability

Controlled growth. To isolate the effect of the representation, we generated Makefiles with k independent options in four families: k computed-name appends (obj-\$(CONFIG_A_{*i*}) += a_{*i*}.o); k ifeq blocks that each append to obj-y; k ifeq/else blocks that set a variable used in an object name; and k ifeq blocks that append to a helper list later appended to obj-y. For k from 1 to 128 we ran KFOLD, the forking executor that KFOLD’s per-word representation replaced (the same code

Table 6. Linux per-file target lists: `KFOLD` versus GNU Make under the same normalized configurations. Each traversed Makefile is evaluated on its own.

Profile	KFOLD active	Make active	TP	FP	FN
Tiny i386	922	922	922	0	0
Defconfig	1,837	1,837	1,837	0	0
Debian	6,201	6,201	6,201	0	0
Allmodconfig	12,605	12,605	12,605	0	0

Table 7. Controlled growth: wall time in seconds on generated Makefiles with k independent options, one run each, five-minute limit (“—”). The forking column also gives the number of states. *KMAX additionally reports k objects with unexpanded names.

Family	k	KFOLD	Forking (states)	KMAX -u	KMAX SMT
append	8	0.63	0.51 (16)	0.68	0.71
append	32	0.62	0.55 (64)	0.70	0.99
append	128	0.63	0.62 (256)	0.80	6.73
ifeq	16	0.62	1.50 (17)	0.70	0.71
ifeq	64	0.62	265.79 (65)	0.75	0.82
ifeq	128	0.62	—	0.81	0.94
value	8	0.62	3.18 (2,048)	0.69*	0.71*
value	12	0.62	75.51 (49,152)	0.70*	0.72*
value	128	0.73	—	0.90*	1.10*
indirect	12	0.62	8.65 (24,577)	0.68	0.70
indirect	16	0.62	184.59 (524,289)	0.69	0.71
indirect	128	0.62	—	0.82	0.93

base before that change, which forks at each conditional and merges identical paths), and KMAX with both of its output modes. Each run had a five-minute limit.

Table 7 shows that `KFOLD`’s cost does not grow with k in any family: every run takes 0.61–0.73 s, most of it interpreter start-up, and 65–67 MB, and every run reports exactly the expected objects. The forking executor’s cost depends on how many distinct values the conditionals create. When conditionals only append distinct words to one list (*ifeq*), it merges them into $k + 1$ states, yet its time still grows to 266 s at $k = 64$ and exceeds the limit at $k = 128$. When each conditional sets a value used in a name (*value*) or extends a helper list (*indirect*), the states multiply: 49,152 states and 76 s at $k = 12$ for *value*, and 524,289 states, 185 s, and 1.7 GB at $k = 16$ for *indirect*; the next sizes time out. On *append* its paths are cheap, because each computed-name assignment forks only on one option. KMAX stays below one second in its text output mode (-u), and its SMT output mode, which `kmaxall` uses, grows to 6.7 s at $k = 128$ on *append*. On *value*, both modes report k objects that no build can produce, such as `c0_$(V0).o` with condition true, besides the $2k$ correct ones.

Whole trees. Table 8 gives `KFOLD`’s cost on the five subjects. The Linux x86 analysis traverses 2,022 Makefile instances, splices 118 includes, and yields 29,245 object paths in 34.0 s with a peak resident size of 162 MB and 82,405 Z3 checks; the rule closure and built-in routes are included. The four other subjects take 0.27–8.0 s. For comparison, KMAX needed 5,174 s for the same Linux tree and exceeded a 12 GB memory limit on three directories (Section 5.7).

Table 8. `KFOLD` runs on the five subjects. Paths include composite members, program objects, and rule prerequisites. Each time is one run on the machine of Section 5.3; memory is peak resident size.

Subject	Makefile instances	Object paths	Includes spliced	Time (s)	Memory (MB)
BusyBox	29	609	0	0.27	66
coreboot	458	4,594	0	7.98	90
Barebox	160	1,436	0	0.96	92
Das U-Boot	289	3,209	0	3.02	95
Linux v6.6 x86	2,022	29,245	118	34.05	162

Table 9. `KMAX` and `KFOLD` against the same Linux builds: precision / overlap (%) and compiled extracted objects that are not predicted (FN_U). “Local” uses `KMAX`’s per-Makefile conditions; “with dirs” conjoins the conditions it records for ancestor directories.

Profile	KFOLD		KMAX local			KMAX with dirs	
	P / O	FN_U	P / O	FN_U		P / O	FN_U
tinyconfig	100.0 / 100.0	0	31.7 / 81.2	21		96.1 / 59.9	125
defconfig	100.0 / 100.0	0	73.8 / 63.2	918		82.6 / 56.5	1,111
Debian	100.0 / 100.0	0	93.7 / 58.7	5,472		95.6 / 56.7	5,782
allmodconfig	100.0 / 100.0	0	96.5 / 64.2	7,479		97.6 / 62.6	7,902

5.7 RQ4: Comparison with `KMAX`

We ran `KMAX` v4.10 the way it analyzes a whole tree: `kmaxall` in tristate mode from Linux’s root `Kbuild` with `SRCARCH=x86`, plus `lib/`. As distributed, `kmaxall` stops the whole run with a Python 3 `TypeError` when any directory fails; it did so after 37 directories, at `fs/hostfs/`. We ran a copy with that one error-reporting call fixed. `KMAX` then analyzed 2,016 directories in 5,174 s. Three directories, `sound/soc/codecs/`, `drivers/mfd/`, and `drivers/i2c/busses/`, raised `MemoryError` under a 12 GB address-space limit that we set to protect the machine, and `fs/hostfs/` failed with an internal `KeyError`; their objects are absent from `KMAX`’s output. `KMAX` reports 28,377 object paths and 29 further paths with unexpanded references, such as `arch/x86/kernel/apic/probe_$(BITS).o`, which no build can produce. Its output gives each object a condition relative to its own Makefile and records each subdirectory’s condition separately; we evaluated both the local conditions and conditions conjoined with every ancestor directory’s recorded condition.

Table 9 compares the two tools on the same four Linux builds. `KMAX`’s local conditions predict too many objects on small configurations: on `tinyconfig`, 855 of its 1,252 predictions are not compiled, most of them in directories that the build never enters. Conjoining the directory conditions removes most of these false positives but loses the whole `arch/x86/` subtree, because `KMAX` records a condition for `arch/x86/` that is a disjunction of `=m` values of unrelated options such as `CONFIG_BLOCK` and is false in all four configurations. In both variants, `KMAX` misses between 21 and 7,902 compiled objects that it extracts. We checked which of these are composite members in `KFOLD`’s analysis: 882 of 918, 5,425 of 5,472, and 7,428 of 7,479 on the three x86-64 profiles with local conditions. `KMAX` gives a composite’s members the condition that the container’s option is `y`, so members of a composite built as a module are never predicted; `fs/ext2/symlink.o`, for example, requires `CONFIG_EXT2_FS=y`, while `allmodconfig` sets it to `m`. On `BusyBox`, the two tools agree exactly: both extract 609 objects and predict the same 162 compiled objects with no false positive; `KMAX` takes 25.6 s and `KFOLD` 0.27 s.

Table 10. Ablation: precision / overlap (%) of the physical comparison with one design element removed. “Earlier entry settings” are the entry settings used before this study. Include splicing, the rule closure, and the built-in semantics do not change the non-Linux results.

Subject / profile	Full <code>KFOLD</code>	No members	No includes	No rules	No built-in sem.	Earlier entry
Linux <code>tinyconfig</code>	100 / 100	100 / 88.5	100 / 100	100 / 91.6	99.0 / 100	98.6 / 43.4
Linux <code>defconfig</code>	100 / 100	100 / 49.4	100 / 100	100 / 97.8	95.1 / 100	94.8 / 81.8
Linux Debian	100 / 100	100 / 38.2	100 / 92.1	100 / 99.5	99.3 / 100	99.4 / 94.7
Linux <code>allmodconfig</code>	100 / 100	100 / 47.4	100 / 95.3	100 / 99.7	99.6 / 100	99.6 / 96.0
Barebox	99.0 / 99.0	99.0 / 96.6	—	—	—	98.9 / 91.6
Das U-Boot (archived)	92.5 / 84.8	92.3 / 82.3	—	—	—	92.3 / 81.7
coreboot	80.9 / 64.8	80.9 / 64.8	—	—	—	77.2 / 36.2

5.8 RQ5: Ablation

We removed one design element at a time and repeated the physical comparison (Table 10). Composite members matter most: without them, `KFOLD` extracts only target words, and overlap falls to 38.2–49.4% on the three x86-64 Linux profiles, because most compiled Linux objects are members. Without the rule closure, the boot image, real-mode, `vDSO`, and purgatory objects are missing, which lowers overlap on `tinyconfig`, where they are a large share of a small kernel, to 91.6%. Without the built-in semantics of Section 3.8, overlap stays at 100% but precision falls to 95.1–99.6%, because built-in composite containers and the built-in objects of directories reached only through `obj-m` are predicted but never written. Without include splicing, overlap on Debian and `allmodconfig` drops to 92.1% and 95.3%, mostly in the AMD and Nouveau GPU drivers, whose Makefiles list their objects in included files. With the entry settings used before this study, which started Linux at the root `Kbuild` without `SRCARCH`, `BITS`, `lib/`, or the boot goal, overlap on `tinyconfig` falls to 43.4%, because most of that small kernel lies under `arch/x86/` and `lib/`. On the other subjects, only composite members and entry settings change the results: the earlier coreboot settings reached 29 Makefiles instead of 458, and the earlier U-Boot settings miss 12 compiled USB objects.

5.9 Threats to validity

Construct validity. Precision and overlap compare path names; neither measures whether a source file builds under an arbitrary configuration. The comparison is limited to the top-level directories that each archive holds, and the one Linux prediction outside that scope was checked with a separate build.

Internal validity. We developed several of `KFOLD`’s semantics, including include splicing, word lists, deferred appends, substitution references, the built-in semantics, and the rule closure, while examining discrepancies on these four Linux configurations, so exact agreement partly reflects fitting to them. Each addition implements a documented behavior of GNU Make or of `Kbuild`’s `Makefile.build` and has a regression test on a small Makefile, and the per-file GNU Make reference is independent of the builds; configurations outside the four evaluated ones and other kernel versions would test generalization. The entry settings copy directory lists and goals from top-level Makefiles by hand, and a copying error would change the traversal. The comparison depends on path normalization, stage prefixes, and the assumption that each archive corresponds to its configuration; fresh builds that reproduced the path sets for two Linux profiles and Barebox reduce that risk for those inventories.

External and conclusion validity. Each non-Linux system contributes one configuration, and Linux four, all for one architecture family. Only two Linux inventories and Barebox were reproduced by

fresh builds, and the U-Boot and coreboot builds did not produce final artifacts. Each measurement is a single run. The results describe this implementation on these subjects and configurations; they do not show that `KFOLD` handles every Kbuild dialect or kernel version.

6 Discussion

6.1 Using a condition

The most direct use of a condition is explanation: a maintainer can see which Makefile expressions, directory routes, and rules cause an object to be built under a proposed assignment. `KFOLD` records, for every object path, the Makefiles and variables or rules that contribute it; for example, it reports that the boot object `compressed/kaslr.o` is a prerequisite of the compressed kernel image and is built under `CONFIG_RANDOMIZE_BASE`. Because the Linux predictions equal the builds, the conditions can also answer the reverse question without a build: which objects a configuration change adds or removes. The web interface exposes both queries.

6.2 Kconfig feasibility and downstream uses

A Kbuild condition and a Kconfig-feasible condition answer different questions. Let $K(c)$ mean that a normalized configuration c satisfies the project's configuration rules. A buildable witness for object o requires at least $K(c) \wedge \Phi_M(o)[c]$ and a successful build. `KFOLD` computes $\Phi_M(o)$ but not K . Configuration sampling, dead-target detection, and minimal reproducing configurations all need both: an unsatisfiable $\Phi_M(o)$ may reflect an unresolved include or the option domain rather than a dead object, and a satisfying assignment still has to pass the configurator. Combining Φ_M with a Kconfig encoding such as `Kclause` [14] is a direct next step.

6.3 Constructs outside the model

The syntax census gives a sense of how often the boundaries in Table 2 arise. Over the five traversals it records 92,318 occurrences, of which 91,271 fall in the modeled categories, 1,002 are recipes and directives outside the model, and 45 are unsupported; Linux accounts for 60,554. The labels come from the parser and classifier, not from semantic checks: the 15 Linux shell calls are labeled modeled although `KFOLD` runs them concretely. The two unresolved coreboot includes and the one unparseable coreboot file show that constructs outside the model remain, and the non-Linux subjects would need the equivalent of Section 3.8 for their own build rules, such as coreboot's stage images.

6.4 Next steps

Three extensions follow from the evaluation. First, `KFOLD` could execute the directory lists and goals of top-level Makefiles instead of taking them from settings; this requires modeling the `include` of architecture Makefiles and the top-level rules that the settings now mirror by hand. Second, the build semantics of Section 3.8 could be generalized to the other dialects, whose settings currently leave them off. Third, a conformance suite of small Makefiles, compared with GNU Make over all Boolean and tristate settings of their options, would test each construct in isolation, including `?=` and `:=`.

7 Related Work

The closest work extracts configuration conditions from Kbuild Makefiles; neighboring work represents variability in source code or symbolic states, and a third line models Kconfig itself.

7.1 Static analysis of Kbuild and build variability

KMAX [3] is the closest prior analysis. It evaluates Kbuild Makefiles with a family-based static analysis whose abstract values are sets of complete string definitions tagged with reaching configurations, represented as BDDs, with the definitions of one variable mutually exclusive; an append combines previous and new definitions by cross-product. It handles computed variable names, includes, and subdirectories, and its evaluation accounts for every C file in the Linux v3.19 and BusyBox 1.25.0 trees. KFOLD shares the idea of carrying configuration conditions on values but guards each word, which keeps k independent appends at k entries and discards order. It also models build semantics beyond the target lists, such as modular composites, built-in routes, and rule prerequisites, and it is validated differently: we evaluate each formula under concrete configurations against GNU Make and real builds. On current Linux, KMAX's whole-tree driver takes 86 minutes, exceeds 12 GB on three directories, and misses the members of modular composites, while KFOLD takes 34 s and matches the builds (Section 5.7).

Nguyen and Nguyen [13] symbolically execute Linux Kbuild Makefiles by forking a path state at each conditional. KFOLD replaces this forking with the per-word guards and branch merge of Algorithm 2, and adds the conditional directory worklist, include splicing, composite members, and build semantics; Section 5.6 measures the difference in state growth. Later work uses Kbuild constraints to localize compilation errors and find unmet dependencies [5, 14]; these applications consume build conditions like those KFOLD produces and combine them with Kconfig constraints.

General build-system analyses predate the Kbuild-specific tools. SYMAKE analyzes Makefiles to support transformations, and MAKAO supports build-system analysis and re-engineering [1, 15]. They target build structure and dependencies rather than per-configuration object selection.

7.2 Variability and symbolic-state representations

Variability-aware parsing and analysis keep alternatives across configurations instead of selecting one preprocessor branch at a time. TypeChef, SuperC, and related work show how such representations support reasoning about highly configurable C code [4, 6, 7, 10]. Kbuild selection is complementary: a source-level analysis still needs to know which translation units enter a build. The build language also has computed variable names, deferred expansion, directory traversal, and rules, so its representation needs differ from those of the C preprocessor.

Symbolic execution and state merging provide the broader design context [8, 9]. Merging avoids one environment per branch combination but moves cost into the symbolic expressions and the merge itself; KFOLD's rule of keeping an unchanged guard unchanged at a merge is what keeps that cost from growing with the number of conditionals.

7.3 Configuration models and validation

Kconfig is a separate variability language. Work on parsing Kconfig and on configuration defects shows why a Makefile-only condition cannot by itself establish feasibility or dead code [2, 12, 16]. Sampling research studies how small sets of configurations can exercise a configurable system [11]. KFOLD's formulas could serve such workflows as the build-side input, once combined with a Kconfig model.

8 Conclusion

KFOLD extracts a build condition for every object path in a Kbuild tree by attaching guards to individual words, merging conditional arms into one state, splicing included fragments, propagating only uncovered guards through the directory graph, and following Kbuild's build semantics through composites, built-in routes, and rules. The representation stays linear where a forking

executor explodes, and it analyzes the Linux x86 tree in 34 s. Evaluated under four concrete Linux configurations, its predicted objects are exactly the compiled ones, and its per-file target lists agree with GNU Make on every target. Its main limitation is that top-level orchestration enters through settings copied from the top-level Makefiles; executing those Makefiles, carrying the build semantics to other dialects, and combining the conditions with Kconfig constraints are the next steps toward configuration synthesis.

References

- [1] Bram Adams, Herman Tromp, Kris De Schutter, and Wolfgang De Meuter. 2007. MAKAO: An extensible framework for analyzing and re-engineering build systems. In *Proceedings of the 2007 IEEE International Conference on Software Maintenance (ICSM)*. 517–518.
- [2] Christian Dietrich, Reinhard Tartler, Wolfgang Schröder-Preikschat, and Daniel Lohmann. 2012. A Robust Approach for Variability Extraction from the Linux Build System. In *Proceedings of the 16th International Software Product Line Conference (SPLC)*.
- [3] Paul Gazzillo. 2017. Kmax: Finding All Configurations of Kbuild Makefiles Statically. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*. 279–290. doi:10.1145/3106237.3106283
- [4] Paul Gazzillo and Robert Grimm. 2012. SuperC: Parsing all of C by taming the preprocessor. In *Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 339–350.
- [5] Paul Gazzillo, Ugur Koc, ThanhVu Nguyen, and Shiyi Wei. 2018. Localizing Configurations in Highly-Configurable Systems. In *Proceedings of the 22nd International Systems and Software Product Line Conference (SPLC), Challenge Track*. doi:10.1145/3233027.3236404
- [6] Christian Kästner, Paolo G. Giarrusso, Tillmann Rendel, Sebastian Erdweg, Klaus Ostermann, and Thorsten Berger. 2011. Variability-Aware Parsing in the Presence of Lexical Macros and Conditional Compilation. In *Proceedings of the ACM International Conference on Object Oriented Programming Systems Languages and Applications (OOPSLA)*. 805–824.
- [7] Andy Kenner, Christian Kästner, Steffen Haase, and Thomas Leich. 2010. TypeChef: Toward Type Checking #ifdef Variability in C. In *Proceedings of the 2nd International Workshop on Feature-Oriented Software Development (FOSD)*. 25–32. doi:10.1145/1868688.1868693
- [8] James C King. 1976. Symbolic execution and program testing. *Commun. ACM* 19, 7 (1976), 385–394.
- [9] Volodymyr Kuznetsov, Johannes Kinder, Stefan Bucur, and George Candea. 2012. Efficient state merging in symbolic execution. In *Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 193–204.
- [10] Jörg Liebig, Sven Apel, Christian Lengauer, and Christian Kästner. 2011. Variability-aware static analysis at scale. In *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE)*. 105–115.
- [11] Flavio Medeiros, Marcio Ribeiro, Rohit Gheyi, Sven Apel, Christian Kästner, Bruno Ferreira, and Karine Santos. 2016. A comparison of 10 sampling algorithms for configurable systems. In *Proceedings of the 38th International Conference on Software Engineering (ICSE)*. 643–654.
- [12] Sarah Nadi, Thorsten Berger, Christian Kästner, and Krzysztof Czarnecki. 2014. Where do configuration defects come from? An empirical study of Linux Kernel configuration bugs. In *Proceedings of the 36th International Conference on Software Engineering (ICSE)*. 677–688.
- [13] ThanhVu Nguyen and KimHao Nguyen. 2020. Using Symbolic Execution to Analyze Linux KBuild Makefiles. In *Proceedings of the 2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 712–716. doi:10.1109/ICSME46990.2020.00078
- [14] Jeho Oh, Necip Fazıl Yıldırım, Julian Braha, and Paul Gazzillo. 2021. Finding Broken Linux Configuration Specifications by Statically Analyzing the Kconfig Language. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. doi:10.1145/3468264.3468578
- [15] Ahmed Tamrawi, Tung Thanh Nguyen, Jafar M Al-Kofahi, and Tien N Nguyen. 2012. SYMAKE: A tool for analyzing and transforming Makefiles. In *Proceedings of the 2012 ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*. 1–4.
- [16] Reinhard Tartler, Daniel Lohmann, Julio Sincero, and Wolfgang Schröder-Preikschat. 2011. Feature Consistency in Compile-Time-Configurable System Software: Facing the Linux 10,000 Feature Problem. In *Proceedings of the Sixth European Conference on Computer Systems (EuroSys)*. 47–60.

A Reproduction Map

Each reported number has a source file. `results/physical_validation.json` contains, for every subject and configuration, the analysis time, memory, census, include statistics, parse errors, predictions outside the archive scope, and the full false-positive, in-set miss, and outside path lists for the physical comparison, both with and without composite members. `results/linux_gnu_make_validation.json` contains the per-file GNU Make comparison, `results/bench_scaling.json` the controlled scaling runs, `results/kmax_physical_validation.json` and `results/kmaxall_linux_x86.meta.json` the KMAX runs, `results/kmax_fn_classes.json` the classification of KMAX's misses, and `results/ablation.json` the ablation. `evidence/manifest.json` records source revisions, archive hashes where available, configuration hashes, and build status.

The scripts are `tools/evaluate_physical.py` (physical comparison), `tools/compare_kfold_with_gnu_make.py` (per-file GNU Make reference), `tools/bench_scaling.py` (controlled growth; it extracts the forking executor from the repository history), `tools/run_kmaxall.py`, `tools/evaluate_kmax_physical.py`, and `tools/classify_kmax_misses.py` (KMAX), `tools/run_ablation.py`, `tools/check_merge_equivalence.py`, `tools/show_makefile.py`, which prints the guarded variables of one Makefile, and `tools/build_web_data.py`, which prepares the web interface in `web/`.

The Linux configuration hashes in the manifest are SHA-256 values of the `.config` bytes. The i386 `tinyconfig` hash begins 377795787624; x86-64 `defconfig` begins e0eef62fcde3; the Debian configuration begins 8fb22042e81e; `allmodconfig` begins 286635003641. A replay that omits the two local Makefile edits or changes the architecture of `tinyconfig` is not the same comparison.